Student Performance Analytics System

Python mini project | NumPy and Pandas

Student details	Information
Prepared for	Mohan Arun R.V. - review course-registration spelling
Degree / college	B.Tech AI & Data Science; Paavai College of Engineering
Course	EWB Courses - Python with AI
Student / register ID	Add your actual ID before submission
Assignment deadline	15 October 2026

Objective

Read a student marks sheet, calculate individual academic results, and generate useful class, subject and department summaries. The program makes repetitive calculations consistent and presents the results in readable reports.

- Calculate totals, averages, grades and marks-based Pass/Fail results.

- Identify highest/lowest performance, top passed students and subject trends.

- Flag low attendance, validate input and save reusable analysis outputs.

Technologies actually used

Tool	Role in the project
Python	Functions, loops, conditions, lists, dictionaries and CLI execution
NumPy	Numerical totals/means, all-subject pass checks, minima and maxima
Pandas	CSV input, DataFrames, filtering, grouping, ranking and CSV export
unittest	24 automated checks plus boundary subcases

Tested: Python 3.12.14, NumPy 2.3.5 and Pandas 2.2.3. PDF formatting uses ReportLab outside the analysis program.

4. Dataset and calculation rules

The input contains 25 synthetic records, 9 required columns, 5 subjects and 4 departments. Values are for demonstration, not real examination results. Department sizes: AI&DS 7, CSE 6, ECE 6, MECH 6.

Field	Meaning / accepted value
Student_ID	Unique nonempty text; leading zeros preserved
Name / Department	Nonempty text; whitespace at edges removed
Python / Mathematics	Numeric subject marks from 0 to 100
Statistics / Computer_Science / English	Numeric subject marks from 0 to 100
Attendance	Numeric percentage from 0 to 100

Marks, results and attendance

- Total = sum of the five subject marks, maximum 500.
- Average = total / 5. Every subject has the same weight.
- Pass requires at least 40 in EVERY subject. A single mark below 40 means Fail.
- Attendance below 75% is Low; otherwise Sufficient. It is a separate flag.

Grade	Condition
A+	Pass and average >= 90
A	Pass and 80 <= average < 90
B+	Pass and 70 <= average < 80
B	Pass and 60 <= average < 70
C	Pass and 50 <= average < 60
D	Pass and 40 <= average < 50
F	Any failed subject, regardless of average

These are project assumptions, not an official institutional grading policy. Decisions use unrounded averages; displays and numerical exports use two decimal places. Rank includes all students; the top list includes passed students only.

Ties keep competition ranks (1, 1, 3, ...). All passed students tied at the requested top-N cutoff are included, so more than N rows may appear.

5. Implementation

main.py coordinates the work. functions.py contains reusable operations so calculations are easy to read, change and test.

Function	Responsibility
load_data / validate_data	Read UTF-8 CSV; check columns, IDs, numeric ranges and missing values
assign_grade	Apply the grade thresholds; failed subjects override average
process_students	Add total, average, result, grade, failed subjects, attendance flag and rank
subject_analysis	Find each subject's highest, lowest, average and pass rate
department_analysis	Group students by department and summarize marks and attendance
top_performers	Select passed students by average while preserving cutoff ties
class_summary	Count passes/failures and find averages, extremes and best subjects
build_report / save_outputs	Display clear analysis and write seven CSVs plus a text report

The main NumPy calculations

```
marks = result[SUBJECTS].to_numpy(dtype=float)
result["Total"] = np.sum(marks, axis=1)
result["Average"] = np.mean(marks, axis=1)
passed = np.all(marks >= PASS_MARK, axis=1)
```

The sample marks array has shape (25, 5): 25 students and 5 subjects. axis=1 combines subjects within each student's row. np.all returns True only if all subject checks in that row are True.

Processing sequence

1. Parse arguments and load data. 2. Validate required values. 3. Compute student results. 4. Analyze subjects and department groups. 5. Select leaders and class metrics. 6. Display and export the reports.

Data quality

Missing columns, empty records, invalid or out-of-range marks, blank identifiers and repeated IDs produce helpful errors. No missing mark is silently invented. Additional input columns are ignored. CSV row numbers include the header row.

6. Features and 7. verified results

Required features include totals/averages, grades, pass/fail counts, subject summaries, class extremes and top performers. Additional features include department comparisons, failed-subject names, attendance flags, validation and eight saved reports.

Metric	Verified sample result
Students	25
Passed / failed	20 / 5
Pass percentage	80.00%
Class average	71.44 / 100
Highest average	Priya and Sneha, 95.20
Lowest average	Vignesh, 34.80
Best subject	English, 75.16 average
Weakest subject	Python, 68.96 average
Average attendance	84.40%
Low attendance	5 students

Top five passed students

Rank	Name	Total / 500	Avg / 100	Grade
1	Priya	476	95.20	A+
1	Sneha	476	95.20	A+
3	Divya	474	94.80	A+
4	Arun Kumar	450	90.00	A+
4	Aishwarya	450	90.00	A+

Examples that explain the rules

Arun Kumar: 92 + 88 + 90 + 94 + 86 = 450; average 90.00; Pass; A+.

Manoj: average 77.00, but Mathematics is 35; Fail and F. Prakash: exactly 40 in every subject and 75% attendance; Pass, D, Sufficient. Sanjay: passes every subject but has 74% attendance; Pass with a Low flag.

7. Output tables

Actual student results from the supplied dataset. The CSV contains additional input columns, failed-subject names and attendance status.

ID	Name	Total	Average	Grade	Result	Rank
S001	Arun Kumar	450	90.00	A+	Pass	4
S002	Priya	476	95.20	A+	Pass	1
S003	Kavin	380	76.00	B+	Pass	12
S004	Nandhini	444	88.80	A	Pass	6
S005	Dinesh	300	60.00	B	Pass	19
S006	Meena	429	85.80	A	Pass	7
S007	Surya	297	59.40	F	Fail	20
S008	Revathi	364	72.80	B+	Pass	14
S009	Harish	240	48.00	D	Pass	22
S010	Divya	474	94.80	A+	Pass	3
S011	Balaji	206	41.20	F	Fail	23
S012	Keerthana	410	82.00	A	Pass	9
S013	Naveen	343	68.60	B	Pass	15
S014	Aishwarya	450	90.00	A+	Pass	4
S015	Prakash	200	40.00	D	Pass	24
S016	Lakshmi	320	64.00	B	Pass	17
S017	Manoj	385	77.00	F	Fail	11
S018	Sneha	476	95.20	A+	Pass	1
S019	Sanjay	287	57.40	C	Pass	21
S020	Pooja	372	74.40	B+	Pass	13
S021	Vignesh	174	34.80	F	Fail	25
S022	Janani	422	84.40	A	Pass	8
S023	Gokul	318	63.60	B	Pass	18
S024	Anitha	392	78.40	B+	Pass	10
S025	Rahul	321	64.20	F	Fail	16

Subject-wise performance

Subject	High	Low	Average	Pass	Fail	Pass %
Python	96	31	68.96	21	4	84%
Mathematics	95	28	69.20	22	3	88%
Statistics	96	35	70.72	24	1	96%
Computer Science	98	38	73.16	24	1	96%
English	97	40	75.16	25	0	100%

The full terminal analysis is also saved as output/analysis_report.txt. The PDF uses actual output tables; no simulated screenshot is presented.

8. Outcome and 9. learning

Verification

All 24 automated tests passed in the tested environment. Multiple boundary subcases are included within several tests. Final checks also treat Python warnings as errors.

- Hand-calculated totals and averages; exact grade/pass boundaries.

- High average with a failed subject; unrounded grading; attendance at 74/75%.

- Tied ranks and top cutoffs; class, subject and department statistics.

- All students passing or failing; invalid values, empty files and duplicate IDs.

- CSV leading zeros/quoted names; command-line behavior and report export.

Implementation issues handled

Issue	Resolution / learning
A failed subject hidden by the average	Check every subject before passing or grading
Incomplete or invalid marks	Reject bad input; correct the CSV rather than fill scores
Equal averages	Keep shared competition ranks and cutoff ties
Rounding near a threshold	Use full precision for grades and ranks
Identifiers such as 001	Read identifying values as text
Running from another folder	Resolve default input/output relative to main.py

Final outcome

The complete local project reads and validates the dataset, computes all required metrics, answers practical analysis questions and exports clear results. Code, data, README, editable documentation, a study guide and sample outputs are included. GitHub publication and form submission remain pending.

Learning and limitations

The project demonstrates functions, loops, conditions, DataFrames, array axes, filtering, grouping, dictionaries and exports. Personalize the editable learning section with what you learned while reviewing and adapting the code.

This is descriptive analysis of one dataset. It does not predict future results or establish that attendance causes performance. Charts, multiple examinations and configurable grading are possible future improvements.

Appendix A. Run and explain

Windows setup

Extract the ZIP. Open the extracted student-performance-analytics folder in VS Code, then open a terminal inside that folder. Use Python 3.11 or later.

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe main.py
```

If Windows recognizes py instead of python, use py -m venv .venv for the first command. The commands call the virtual environment directly, so activation is unnecessary. Package installation needs internet; analysis runs offline afterward.

Tests and optional arguments

```
.\.venv\Scripts\python.exe -m unittest discover -s tests -v
.\.venv\Scripts\python.exe main.py --top 3
.\.venv\Scripts\python.exe main.py --output custom_reports
```

Files you should know

File / folder	Purpose
main.py / functions.py	Entry point and reusable calculations
students.csv / requirements.txt	Input dataset and two analysis dependencies
output/	Seven CSV reports plus analysis_report.txt
docs/	Documentation, explanation/viva guide and submission steps
tests/test_project.py	24 automated checks

A simple presentation explanation

"My program reads student data using Pandas. NumPy calculates totals and averages. Functions apply the grades and subject-based pass rule. I compare subjects and departments, identify top passed students and flag low attendance. The program displays results and saves eight reports."

Practice before the review

- Explain why Manoj fails despite a 77.00 average.

- Run --top 1 and explain why both Priya and Sneha appear.

- Change a mark to 39 or attendance to 74 in a copy, predict the result, then run.

Read docs/EXPLAIN_AND_VIVA.md for 22 simple viva answers and a five-minute demonstration plan.

10. GitHub and final submission

Detail	Status / action
Student / register ID	Pending: add your actual course or college ID
Final repository URL	Pending: add the actual URL after repository creation
Google Form URL	Use the form supplied by EWB Courses; no URL is in the assignment PDF
Deadline	On or before 15 October 2026

Submission sequence

1. Run and review the project locally; understand the individual implementation.
2. Review personal details and documentation. Keep the required dataset size.
3. Create a GitHub repository, for example student-performance-analytics.
4. Upload the extracted code, dataset, README, tests, docs and sample outputs.
5. Copy the actual repository URL and add it to the editable documentation.
6. Verify visibility/access as instructed by the trainer.
7. Submit the repository link through the trainer's Google Form.
8. Verify final documentation and repository contents before the deadline.

Follow the trainer's exact workflow. docs/GITHUB_SUBMISSION.md provides browser upload steps and Git commands. This download has not created an online repository or submitted a form.

Final review checklist

- Working Python code; NumPy/Pandas used; functions and loops included.
- Dataset, README, documentation and output reports present.
- Tests pass; personal details and final repository link reviewed.
- Repository uploaded/verified; form confirmation retained.

Learning references

https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html

https://pandas.pydata.org/docs/user_guide/groupby.html

https://numpy.org/doc/stable/reference/generated/numpy.sum.html

https://numpy.org/doc/stable/reference/generated/numpy.mean.html

https://docs.python.org/3/tutorial/controlflow.html